AcquaXcontrol — API v1 (Guia de Integração)

Documento para parceiros e sistemas terceiros. Descreve a API pública do AcquaXcontrol para leitura de condomínios, unidades, medidores, leituras e relatórios de consumo, além de envio de leituras e redefinição de acesso de usuários.

- **Base URL:** `https://www.acquaxcontrol.com.br/api/v1`
- **Protocolo:** HTTPS, JSON (`application/json`)
- **Versão:** v1
- **Status do serviço:** `GET /api/v1`

---

1. Autenticação

A API usa **API Key** enviada no cabeçalho `Authorization`:

```
Authorization: Bearer ak_<uuid-da-chave>_<segredo>
```

O token é gerado pelo administrador no painel do AcquaXcontrol (menu **API**). Ele é exibido **uma única vez**, no momento da criação. Guarde-o em local seguro (gere um novo se perder).

Cada chave define:

Item	Descrição
Permissões	Lista de `recurso:ação` (veja a seção 2)
Validade	Data de expiração opcional (chaves temporárias)
Escopo	Pode ser restrita a **um condomínio** ou a **uma administradora**
IPs permitidos	Lista opcional de IPs/CIDRs autorizados
Status	`active` , `suspended` , `revoked` , `expired`

Escopo: se a chave estiver restrita a um condomínio, todas as consultas retornam apenas dados daquele condomínio, mesmo que a requisição peça outro `complex_id` .

Se a chave for exposta, peça a revogação imediata e gere outra.

2. Permissões e endpoints

Método	Rota	Permissão	O que faz
GET	`/api/v1`	—	Informações da API e status
GET	`/api/v1/complexes`	`complexes:read`	Lista condomínios
GET	`/api/v1/apartments`	`apartments:read`	Lista unidades (apartamentos)
GET	`/api/v1/meters`	`meters:read`	Lista medidores
GET	`/api/v1/readings`	`readings:read`	Lista leituras
POST	`/api/v1/readings`	`readings:create`	Envia leituras (até 200 por requisição)
GET	`/api/v1/reports`	`reports:read`	Relatórios de consumo/ faturamento por unidade e mês
GET	`/api/v1/users`	`users:read`	Lista usuários (moradores, síndicos, administração)
POST	`/api/v1/users/{id}/reset`	`users:update`	Redefine o acesso de um usuário

Permissão ausente na chave → **403**.

3. Convenções

- **IDs:** UUID em texto (ex.: `732eb5c6-2f64-45c3-abad-2f5d8ed95c89`).
- **Datas:** ISO 8601 em UTC (ex.: `2026-10-06T00:00:00.000Z`). O campo `readAtDate` vem no formato `AAAA-MM-DD HH:mm:ss`.
- **Valores monetários:** número decimal em reais (BRL), sem símbolo (ex.: `544.02`).
- **Volumes:** número decimal em m³.
- **Paginação:** parâmetros `take` e `skip`; a resposta traz `meta`:

```
"meta": { "total": 56, "take": 20, "skip": 0, "hasNextPage": true,
"hasPreviousPage": false }
```

Endpoint	`take` padrão	`take` máximo
`/complexes`, `/apartments`, `/meters`, `/readings`, `/users`	20	100
`/reports`	50	200

- **Formato de resposta:** coleções vêm em `{ "data": [...], "meta": {...} }`.

---

4. Endpoints em detalhe

4.1 `GET /api/v1` — status

```
{
  "api": "AcquaX Control API",
  "version": "v1",
  "status": "online",
  "authentication": "Bearer Token (API Key)"
}
```

4.2 `GET /api/v1/complexes` — condomínios

Parâmetros: `search` (nome), `take`, `skip`.

```
{
  "data": [
    {
      "id": "cd7d24e2-329a-460c-afbc-81b32b28632d",
      "socialName": "A4 Offices",
      "street": "Av. das Américas",
      "number": "13.685",
      "city": "Rio de Janeiro",
      "state": "RJ",
      "createdAt": "2025-06-18T13:36:21.000Z",
      "_count": { "blocks": 9 }
    }
  ],
  "meta": { "total": 343, "take": 20, "skip": 0, "hasNextPage": true,
"hasPreviousPage": false }
}
```

4.3 `GET /api/v1/apartments` — unidades

Parâmetros: `complex_id`, `block_id`, `search` (nome da unidade), `take`, `skip`.


```
{
  "id": "c91e31ec-f4f5-49b6-a7d4-b15d2a3dd118",
  "name": "101",
  "status": "Ativo",
  "fraction": 0,
  "blockId": "49044575-5765-45c9-834d-ce0a32c5132c",
  "complexId": "732eb5c6-2f64-45c3-abad-2f5d8ed95c89",
  "companyId": "e8155542-396d-495c-a092-4a7b97fc4ea3",
  "block": { "id": "49044575-5765-45c9-834d-ce0a32c5132c", "name": "1" },
  "meters": [{ "id": "d21fe3e9-dc75-4524-8942-869b5235c5a6", "register":
"RSDCA101", "status": "Ativo" }]
}
```


`fraction` é a fração ideal da unidade (usada em rateios).

4.4 `GET /api/v1/meters` — medidores

Parâmetros: `complex_id`, `block_id`, `search` (referência do medidor), `take`, `skip`.


```
{
  "id": "dc4d55ab-05f9-458a-b058-a33d30814f8e",
  "register": "RSDCA203",
  "status": "Ativo",
  "createdAt": "2025-06-24T16:56:06.000Z",
  "apartment": {
    "id": "3eb4b257-f178-463f-9444-b89cd915e7ad",
    "name": "203",
    "block": { "id": "49044575-5765-45c9-834d-ce0a32c5132c", "name": "1" }
  }
}
```


O campo `register` é a referência do medidor (a mesma que aparece nas planilhas e no aplicativo de campo).

4.5 `GET /api/v1/readings` — leituras

Parâmetros: `meter_id`, `apartment_id`, `complex_id`, `from` (AAAA-MM-DD), `to` (AAAA-MM-DD), `take`, `skip`. Ordenado da leitura mais recente para a mais antiga.

```
{
  "id": "c5d4bf82-a9f6-44dc-b139-f37d9590b91b",
  "reading": 3471.345,
  "readAt": "2026-10-06T00:00:00.000Z",
  "readAtDate": "2026-10-06 00:00:00",
  "monthRef": "Outubro",
  "yearRef": "2026",
  "isPreReading": false,
  "isManualReading": true,
  "nextReadingDate": null,
  "urlCover": "https://cdn.acquaxcontrol.com.br/.../Fotos/708.jpg",
  "meterId": "20acff4d-e83c-4e68-b2f4-ebeb3e656c5b",
  "apartmentId": "29c4487d-1fe4-4b5e-9cea-e8d12d373247",
  "createdAt": "2026-10-13T18:05:21.000Z",
  "meter": {
    "id": "20acff4d-e83c-4e68-b2f4-ebeb3e656c5b",
    "register": "RSDCA708",
    "apartment": { "id": "29c4487d-1fe4-4b5e-9cea-e8d12d373247", "name":
"708", "block": { "id": "...", "name": "1" } }
  }
}
```


> `monthRef` pode vir como número (`"10"`) ou por extenso (`"Outubro"`) em registros antigos. Para filtrar por período use sempre `from` / `to` .

4.6 `POST /api/v1/readings` — enviar leituras

Corpo: um objeto ou `{ "readings": [ ... ] }` (até **200** itens por requisição).


```
{
  "readings": [
    { "meterId": "20acff4d-e83c-4e68-b2f4-ebeb3e656c5b", "reading": 3480.5,
"readAt": "2026-11-05" }
  ]
}
```


Campo	Obrigatório	Descrição
`meterId`	sim	ID do medidor
`reading`	sim	Leitura em m³

Campo	Obrigatório	Descrição
`readAt`	não	Data da leitura (padrão: agora)

Resposta (**201** quando ao menos uma leitura é criada):


```
{
  "created": [
    { "id": "2c2c10fc-9a0c-4462-b4f0-92194fb6eecd", "reading": 3480.5,
"readAt": "2026-11-05T00:00:00.000Z", "monthRef": "11", "yearRef": "2026",
"meterId": "20acff4d-e83c-4e68-b2f4-ebeb3e656c5b" }
  ],
  "errors": [],
  "summary": "1 leitura(s) criada(s), 0 erro(s)."
}
```


Regras:

- Cada medidor aceita **uma leitura por mês/ano**. Se já existir, o item volta em `errors` com o ID da leitura existente e nada é sobrescrito.
- O medidor precisa pertencer ao escopo da chave (`403` /erro por item caso contrário).
- Se todos os itens falharem, o status é **400** e o corpo traz o detalhe em `errors`.

Exemplo de erro por item:


```
{ "created": [], "errors": [{ "index": 0, "error": "Medidor não encontrado:
inexistente" }], "summary": "0 leitura(s) criada(s), 1 erro(s)." }
```


4.7 `GET /api/v1/reports` — relatórios de consumo

Parâmetros **obrigatórios**: `month` (1 a 12) e `year` (AAAA). Opcionais: `apartment_id`, `complex_id`, `take` (máx. 200), `skip`.


```
{
  "id": "2c2c10fc-9a0c-4462-b4f0-92194fb6eecd",
  "monthRef": "10",
  "yearRef": "2026",
  "consumption": 11.675,
  "totalConsumption": 11.675,
  "consumptionCost": 127.13,
  "sewageCost": 127.13,
  "partial": 0,
```

```
  "kiteCarConsumption": 0,
  "kiteCarCost": 0,
  "utilityType": "water",
  "totalUnit": 254.27,
  "apartmentId": "29c4487d-1fe4-4b5e-9cea-e8d12d373247",
  "blockId": "49044575-5765-45c9-834d-ce0a32c5132c",
  "complexId": "732eb5c6-2f64-45c3-abad-2f5d8ed95c89",
  "apartment": { "id": "29c4487d-1fe4-4b5e-9cea-e8d12d373247", "name":
"708", "block": { "id": "...", "name": "1" } }
}
```


Campo	Significado
`consumption` / `totalConsumption`	Consumo da unidade no mês (m³)
`consumptionCost`	Valor do consumo de água
`sewageCost`	Valor do esgoto
`partial`	Rateio de área comum da unidade
`kiteCarConsumption` / `kiteCarCost`	Carro-pipa (m³ e R$) quando houver
`utilityType`	`water`, `gas` ou `energy`
`totalUnit`	Total a pagar pela unidade

4.8 `GET /api/v1/users` — usuários

Parâmetros: `complex_id`, `search` (nome ou e-mail), `take`, `skip`.


```
{
  "id": "4424d634-2cf8-4768-81b1-211b4b9d8386",
  "name": "Administração",
  "email": "elisiacunha35@gmail.com",
  "telephone": null,
  "cell": null,
  "createdAt": "2026-08-13T12:59:48.000Z",
  "updatedAt": "2026-10-08T18:40:21.000Z"
}
```


Com chave sem escopo de condomínio, a lista traz todos os usuários. Com escopo, apenas os vinculados àquele condomínio.

4.9 `POST /api/v1/users/{id}/reset` — redefinir acesso

Permissão `users:update`. Corpo opcional:

```
{ "newPassword": "SenhaForte123" }
```

Resposta:

```
{
  "success": true,
  "user": { "id": "4424d634-...", "name": "Maria Souza", "email":
"maria@exemplo.com", "mustUpdateCredentials": true },
  "tempPassword": "Acquax@7F2K9A",
  "message": "Acesso restaurado. Senha temporária gerada, compartilhe com
segurança."
}
```

- Sem `newPassword`: o sistema gera uma senha temporária e exige troca no próximo login.
- A senha temporária volta no campo `tempPassword` da resposta, para o parceiro entregar ao morador.
- Com `newPassword`: mínimo de **6 caracteres** (menos que isso retorna **400**).
- Não é permitido redefinir o usuário administrador master (`403`).
- Com chave de escopo, só é possível redefinir usuários vinculados ao condomínio.

---

5. Erros

Status	Quando
400	Parâmetros inválidos ou ausentes (ex.: falta `month` / `year` em `/reports`)
401	Token ausente, inválido, expirado ou revogado
403	Permissão insuficiente, IP não autorizado, chave suspensa ou recurso fora do escopo
404	Recurso não encontrado
500	Erro interno (pode ser tentado novamente com espera)

Formato:

```
{ "error": "Token de autenticação não fornecido. Use Authorization: Bearer <token>", "code": "UNAUTHORIZED" }
```

---

6. Exemplos

cURL

```
curl "https://www.acquaxcontrol.com.br/api/v1/readings?from=2026-10-01&to=2026-10-31&take=50" \
  -H "Authorization: Bearer ak_00000000-0000-0000-0000-000000000000_segredo"
```

JavaScript (fetch)

```
const res = await fetch(
  "https://www.acquaxcontrol.com.br/api/v1/reports?month=10&year=2026&complex_id=SEU_COMPLEXO",
  { headers: { Authorization: `Bearer ${process.env.ACQUAX_API_KEY}` } }
);
if (!res.ok) throw new Error(`${res.status} ${await res.text()}`);
const { data, meta } = await res.json();
```

Python (requests)

```
import requests

r = requests.get(
    "https://www.acquaxcontrol.com.br/api/v1/apartments",
    params={"complex_id": "SEU_COMPLEXO", "take": 100},
    headers={"Authorization": f"Bearer {CHAVE}"},
    timeout=30,
)
r.raise_for_status()
print(r.json()["meta"], len(r.json()["data"]))
```

---

7. Boas práticas

- Use paginação (`take` / `skip`) até `hasNextPage` ser `false` ; não peça tudo em uma chamada só.
- Consulte `/reports` por mês e condomínio em vez de varrer todas as leituras.
- Trate `401` como chave inválida/revogada e `429` / `5xx` com nova tentativa exponencial (1s, 2s, 4s...).
- Não há um teto rígido de requisições por minuto hoje; recomendamos no máximo 5 requisições por segundo por chave.
- Cache os cadastros (`complexes` , `apartments` , `meters`): mudam pouco.
- As fotos de medidor (`urlCover`) são servidas por CDN e podem ser baixadas diretamente pela URL.

---

8. Contato

Dúvidas, novas permissões ou aumento de escopo:

- **ti.acquax@gmail.com**
- **adm@acquaxdobrasil.com.br**